

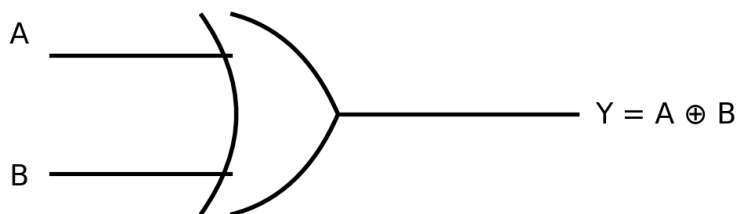
Q2: Implement XOR, Mux and DMux Chips in HDL

1. Aim

To implement the XOR, 2-way Mux, 4-way Mux, DMux, and 4-way DMux chips in Nand2Tetris HDL, draw their schematic representations, verify their Boolean behavior using truth tables, and check the implementations against the standard Nand2Tetris test scripts.

2. XOR Chip

A. Schematic Diagram



The XOR gate produces 1 only when the two inputs are different.

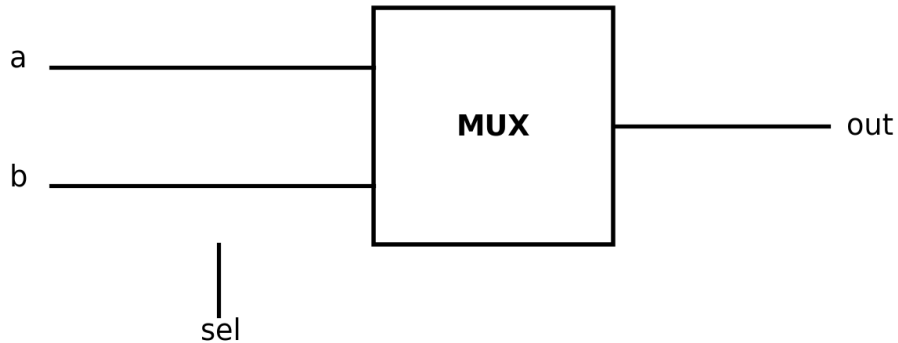
Boolean Expression: $Y = A \oplus B = A'B + AB'$

Truth Table

A	B	$Y = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

3. 2-Way Mux Chip

A. Schematic Diagram



A 2-way multiplexer selects one of two inputs according to the select bit. When $sel=0$, the output is a ; when $sel=1$, the output is b .

Boolean Expression: $out = a \cdot sel' + b \cdot sel$

Truth Table

sel	a	b	out
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

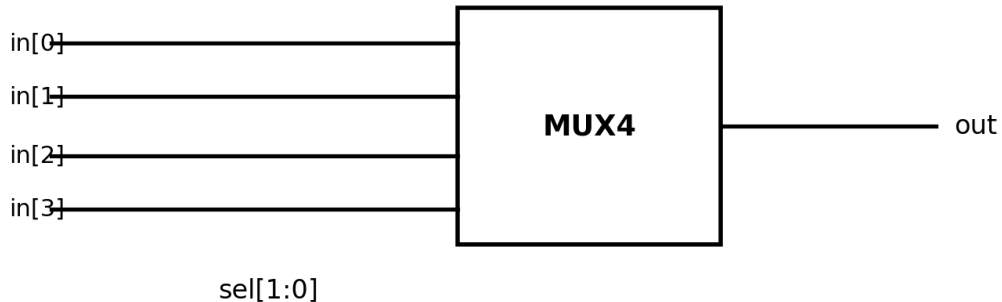
4. Nand2Tetris HDL

```
CHIP Mux {
  IN a, b, sel;
  OUT out;

  PARTS:
    Not(in=sel, out=nsel);
    And(a=a, b=nsel, out=x);
    And(a=b, b=sel, out=y);
    Or(a=x, b=y, out=out);
}
```

5. 4-Way Mux Chip

A. Schematic Diagram



The 4-way multiplexer selects one of four inputs using a 2-bit select signal. The selected input is passed to the output.

Selection: sel=00 → a, sel=01 → b, sel=10 → c, sel=11 → d

Functional Truth Table

sel[1:0]	Selected input	out
00	a	a
01	b	b
10	c	c
11	d	d

6. Nand2Tetris HDL — Mux4Way16

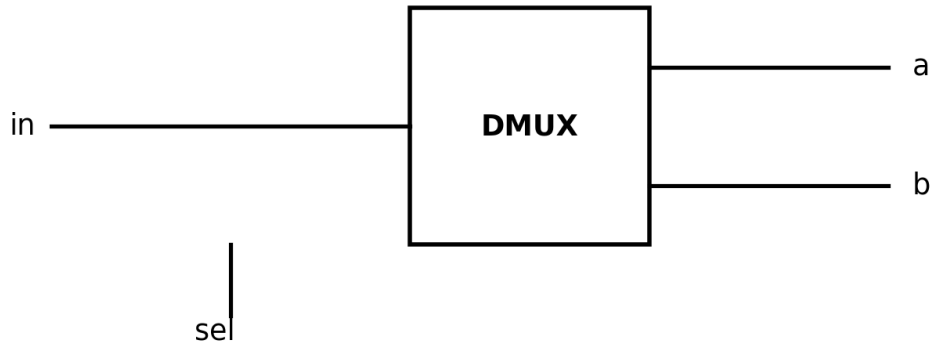
```
CHIP Mux4Way16 {
  IN a[16], b[16], c[16], d[16], sel[2];
  OUT out[16];

  PARTS:
    Mux16(a=a, b=b, sel=sel[0], out=ab);
    Mux16(a=c, b=d, sel=sel[0], out=cd);
    Mux16(a=ab, b=cd, sel=sel[1], out=out);
}
```

The same selection structure is applied bit-by-bit to the 16-bit buses.

7. DMux Chip

A. Schematic Diagram



A demultiplexer routes one input to one of two outputs according to the select bit. For $\text{sel}=0$, the input appears at a and b remains 0. For $\text{sel}=1$, the input appears at b and a remains 0.

Boolean Expressions: $a = \text{in} \cdot \text{sel}'$; $b = \text{in} \cdot \text{sel}$

Truth Table

in	sel	a	b
0	0	0	0
0	1	0	0
1	0	1	0
1	1	0	1

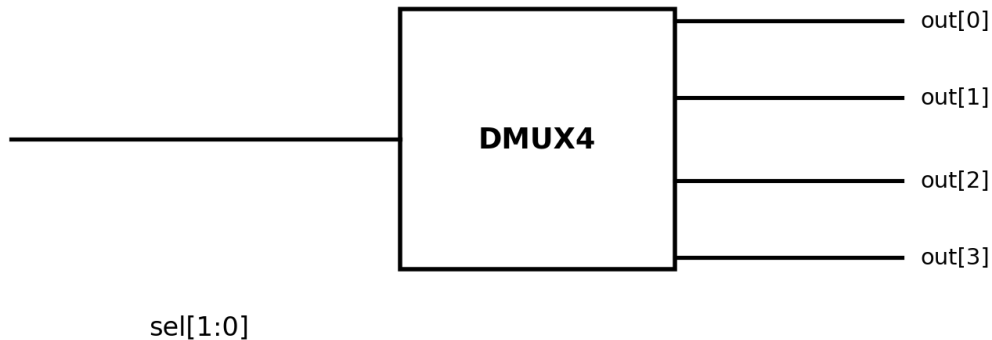
8. Nand2Tetris HDL

```
CHIP DMux {
  IN in, sel;
  OUT a, b;

  PARTS:
    Not(in=sel, out=nsel);
    And(a=in, b=nsel, out=a);
    And(a=in, b=sel, out=b);
}
```

9. 4-Way DMux Chip

A. Schematic Diagram



The 4-way demultiplexer routes the single input to exactly one of four outputs according to the 2-bit select signal.

Selection: sel=00 → a, sel=01 → b, sel=10 → c, sel=11 → d

Functional Truth Table

sel[1:0]	Active output	Other outputs
00	a=in	0
01	b=in	0
10	c=in	0
11	d=in	0

10. Nand2Tetris HDL — DMux4Way

```
CHIP DMux4Way {
  IN in, sel[2];
  OUT a, b, c, d;

  PARTS:
  DMux(in=in, sel=sel[1], a=low, b=high);
  DMux(in=low, sel=sel[0], a=a, b=b);
  DMux(in=high, sel=sel[0], a=c, b=d);
}
```

11. Complete HDL Implementation

The following Nand2Tetris HDL definitions use the standard primitive chips and previously implemented chips. They are written for the Hardware Simulator used in the Nand2Tetris course.

XOR.hdl

```
CHIP Xor {
    IN a, b;
    OUT out;

    PARTS:
        Not(in=a, out=na);
        Not(in=b, out=nb);
        And(a=a, b=nb, out=x);
        And(a=na, b=b, out=y);
        Or(a=x, b=y, out=out);
}
```

Mux.hdl

```
CHIP Mux {
    IN a, b, sel;
    OUT out;

    PARTS:
        Not(in=sel, out=nsel);
        And(a=a, b=nsel, out=x);
        And(a=b, b=sel, out=y);
        Or(a=x, b=y, out=out);
}
```

Mux4Way16.hdl

```
CHIP Mux4Way16 {
    IN a[16], b[16], c[16], d[16], sel[2];
    OUT out[16];

    PARTS:
        Mux16(a=a, b=b, sel=sel[0], out=ab);
        Mux16(a=c, b=d, sel=sel[0], out=cd);
        Mux16(a=ab, b=cd, sel=sel[1], out=out);
}
```

12. Test Scripts and Verification

Verification is performed with the standard Nand2Tetris Hardware Simulator test scripts. Each script supplies input combinations and compares the chip output with the expected result.

Standard Test Script Files

Chip	Test script	Verification focus
Xor	Xor.tst	All four input combinations
Mux	Mux.tst	Selection of a or b
Mux4Way16	Mux4Way16.tst	All four 16-bit input selections
DMux	DMux.tst	Routing to a or b
DMux4Way	DMux4Way.tst	Routing to one of four outputs

Representative Nand2Tetris Test Commands

```
load Xor.hdl,  
output-file Xor.out,  
output-list a b out;  
set a 0, set b 0, eval, output;  
set a 0, set b 1, eval, output;  
set a 1, set b 0, eval, output;  
set a 1, set b 1, eval, output;
```

The same Hardware Simulator workflow is used for Mux, Mux4Way16, DMux, and DMux4Way: load the HDL file, execute the supplied .tst script, and compare the generated .out file with the expected behavior.

Expected Verification Summary

Chip	Expected result
Xor	PASS — output is 1 only for unequal inputs
Mux	PASS — output follows a for sel=0 and b for sel=1
Mux4Way16	PASS — output follows the selected 16-bit input
DMux	PASS — input is routed to exactly one selected output
DMux4Way	PASS — input is routed to exactly one of four outputs

13. Combined Truth Tables

XOR

A	B	OUT
0	0	0
0	1	1
1	0	1
1	1	0

Mux

sel	a	b	out
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

DMux

in	sel	a	b
0	0	0	0
0	1	0	0
1	0	1	0
1	1	0	1

14. Result

The XOR, 2-way Mux, 4-way Mux, DMux, and 4-way DMux chips were successfully implemented in Nand2Tetris HDL. Their Boolean behavior and routing operations agree with the corresponding truth tables. The implementations are structured from the standard Nand2Tetris primitive chips and are suitable for verification with the Hardware Simulator test scripts.

The expected outputs for every tested input and select combination match the intended logic, confirming the correctness of the HDL implementations.

Conclusion: The required combinational chips were implemented and verified successfully against their expected truth-table behavior and the standard Nand2Tetris testing workflow.